

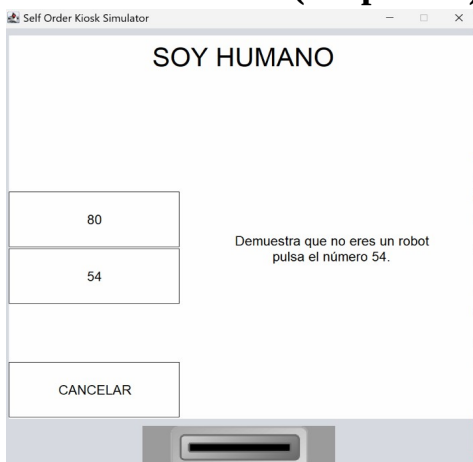
- Tiempo total de la prueba: 1 hora
- Puntuación: 3 puntos que corresponden al 30% de la nota de la asignatura

Modificación de la práctica “Sistema Automático de Venta de Billetes de Cercanías”

Desde el campus virtual debes:

- Descargar la práctica que ya has entregado y descomprimirla en la unidad R para evitar pérdidas de datos.
- Utilizando NetBeans, modificar la práctica según las dos modificaciones que se proponen en el siguiente enunciado.
- Generar un nuevo ZIP con el contenido de todo el proyecto y subirlo a la actividad correspondiente.

Modificación 1 – (1.5 puntos) – Ataque bot



La aplicación de compra de billetes puede estar siendo atacada por bots, por lo que se ha añadido una **ventana de verificación antes de proceder al pago**. En dicha ventana se mostrará un **número aleatorio entre 0 y 100 en la descripción**, junto con **tres botones: Cancelar y dos botones numéricos, uno de los cuales coincidirá con el número mostrado**.

En caso de que el usuario pulse el botón con el **número incorrecto** o el botón **Cancelar**, la aplicación **volverá a la ventana inicial** y se cancelará la venta. Si el usuario pulsa el botón con el **número correcto**, la compra **continuará con normalidad**.

Como ayuda, el siguiente código en Java permite generar números enteros aleatorios entre 0 y 100 y pintarlo por pantalla.

```
import java.util.Random;
...
Random random = new Random();
int numero = random.nextInt(101); // 0 a 100 inclusive
System.out.println("Número generado: " + numero);
```

Modificación 2 – (1.5 puntos) – Compra anticipada

Hasta ahora los billetes no tenían fecha de validez. La empresa de transportes ha decidido incorporar la fecha de validez a los billetes de manera que el billete solo sea válido para la fecha que se indique en el propio billete.

Además, se desea incorporar a la aplicación la posibilidad de comprar un billete de manera anticipada, para un viaje planificado que se realizará mañana.

Para ello, en la pantalla de bienvenida, se propone añadir a los ya existentes el botón de **“Compra anticipada para mañana”**. Al seleccionarlo, la fecha que aparecerá en el billete se retrasará un día respecto a la fecha actual. Luego, el proceso continuará de manera normal, pidiendo la estación de origen, la de destino y finalmente la pantalla de pago.

Tras esta modificación, **la información de validez del billete debe quedar registrada en todos los tickets generados**, siendo los billetes válidos solo para la fecha que aparece en ellos impreso. Los billetes normales con la fecha de hoy y los billetes para mañana con la fecha de mañana.

Como ayuda, el siguiente código en Java permite obtener la fecha de hoy, sumarle un día y pintarlo por pantalla.

```
import java.time.LocalDate;
...
LocalDate hoy = LocalDate.now();
LocalDate dentroDeTresDias = hoy.plusDays(1);
System.out.println(dentroDeTresDias.toString());
```


- Tiempo total de la prueba: 1 hora
- Puntuación: 3 puntos que corresponden al 30% de la nota de la asignatura

El tarjetero electrónico

Se pretende diseñar una aplicación móvil destinada a almacenar todas las tarjetas RFID de un usuario.

Estas tarjetas pueden incluir: tarjetas de crédito, llaves electrónicas para abrir los despachos y aulas de la universidad, tarjetas de fidelización, tarjetas de transporte, etc.

La aplicación permitirá dar de **alta nuevas tarjetas** y **eliminar tarjetas existentes**.



Cada tarjeta poseerá obligatoriamente un **nombre**, una **imagen** y un **código alfanumérico**.

Además, las tarjetas podrán tener opcionalmente un **número ilimitado de campos alfanuméricos** personalizables por el usuario: nombres, zonas, fechas de validez... Estos campos podrán ser rellenados por el usuario para recordar elementos asociados a la tarjeta y podrá darlos de alta, borrarlos y editarlos a voluntad.

Al iniciar la aplicación se permitirá seleccionar cualquiera de las tarjetas almacenadas. Para ello, se **mostrará una lista donde cada tarjeta aparecerá asociada a su imagen representativa** (figura a).

Además, presentará un **botón “+” para añadir nuevas tarjetas**. Si se pulsa dicho botón se presentará una pantalla de edición que permitirá añadir una tarjeta nueva (figura b).

Cuando el usuario seleccione una tarjeta en la pantalla inicial, dicha tarjeta pasará a estar **activa durante 60 segundos**. Durante ese tiempo, la aplicación mostrará el nombre y la imagen de la tarjeta. Debajo mostrará todos los campos opcionales que estén definidos para ella. Abajo, mostrará un botón para dar de baja la tarjeta (figura c).

Para construir las diferentes pantallas se usará la clase `SmartphoneGUI` que ya se proporciona implementada y que dispone de los siguientes métodos:

- **`clearScreen()`**: borra la pantalla.
- **`int showImage(String imageFileName)`**: añade una imagen a la pantalla y devuelve un identificador que se usará para informar de click sobre dicha imagen. La imagen podrá ser la de una tarjeta o la de un botón.
- **`showText(String text)`**: añade un texto en la pantalla.
- **`int showTextInputBox()`**: añade una caja de entrada de texto para que el suario pueda añadir texto y devuelve un identificador a dicha caja.
- **`String readTextInputBox(int id)`**: devuelve el texto introducido por el usuario en la caja de texto correspondiente al identificador.
- **`int waitEvent(int seconds)`**: devuelve el identificador asociado a la imagen clicada, cero si el usuario no ha hecho click en ninguna imagen.
- **`String selectionImageFromGallery()`**: permite obtener el nombre de una imagen desde la galería de la cámara.

Además, tras seleccionar la tarjeta activa la aplicación comenzará a enviar una señal RFID cada 2 segundos con el código asociado a esa tarjeta, utilizando el siguiente método de la clase RFID.

boolean sendCode(String code): método que envía por RFID el código suministrado en el parámetro. Devuelve false si ningún dispositivo ha recibido el código. Devuelve true si alguna aplicación externa ha recibido el código transmitido por RFID.

Si algún dispositivo externo recibe el código o si pasan 60 segundos y ningún dispositivo externo recibe el código la aplicación dejará de transmitir el código por RFID y pasará a mostrar de nuevo la pantalla inicial con todas las tarjetas.

Entrega (en PDF)

Utilizando **StarUML** (u otro programa de modelado) y **Wordpad** (u otro procesador de texto), entregar un fichero PDF que contenga:

a) Clases (1.5 puntos): Texto (máx. 40 palabras por clase) explicando su función. Diagrama de clases con relaciones (herencia, uso, asociación), multiplicidades, propiedades y métodos con visibilidad.

b) Diagrama de secuencia (0.75 puntos): Interacción entre los objetos que muestre la secuencia en la que la aplicación se pone en marcha, muestra la pantalla inicial con todas las tarjetas almacenadas, espera a que se pulse alguna tarjeta o el botón “+” y se pase a la siguiente pantalla.

c) Diagrama de actividad (0.75 puntos): Descripción del proceso general de la aplicación, incluyendo el mostrado de todas las tarjetas en la pantalla inicial, el mostrado de una tarjeta activa y el proceso de transmisión del código por RFID hasta que el proceso concluye con o sin éxito.

Entrega: Subir el PDF generado a la actividad correspondiente en el aula virtual.